

Aktivitas Percobaan

1. Jalankan program, pastikan aplikasi dapat dijalankan dengan baik pada emulator atau perangkat Android fisik. Kemudian klik tombol "Increment" beberapa kali, ambil satu screenshot setelah tombol "Increment" diklik beberapa kali.



- a. Apa yang terjadi setiap kali tombol "Increment" diklik?
Angka pada tampilan bertambah satu setiap tombol diklik
 - b. Apa yang menyebabkan hal tersebut terjadi?
Saat tombol diklik, `onClick = { number++ }` dijalankan dan mengubah nilai number variabel ini dibuat dengan `mutableStateOf`, sehingga Compose mengetahui perubahannya dan melakukan recomposition pada composable yang membaca nilai tersebut. Akibatnya Text menampilkan angka yang baru.
 - c. Variabel *state* apa yang berperan pada Composable Text yang menampilkan perubahan angka pada Text tersebut?
number
 - d. Variabel *state* apa yang berperan untuk menampilkan label teks pada Composable Button yang bertuliskan "Increment" tersebut?
label, yang dideklarasikan dengan `mutableStateOf("Increment")`
2. Pada percobaan tahap sebelumnya, ubah kode program pada Composable `CounterScreen` agar setiap kali tombol "Increment" diklik, nilai yang ditampilkan menjadi increment 2 (bertambah 2 setiap klik). Ubah pula label tombol "Increment" menjadi "Increment 2".
 - a. Perubahan apa yang perlu dilakukan dan dilakukan di mana?
Perubahan dilakukan di dalam composable `CounterScreen`

Kotlin

```
val label by remember { mutableStateOf("Increment 2")  
} // label tombol  
...  
Button(onClick = { number += 2 }) {  
    // nilai bertambah 2  
    Text(text = "$label")  
}
```

- b. Jalankan program dan ambil masing-masing satu screenshot sebelum dan sesudah tombol Increment diklik untuk memperlihatkan bahwa perubahan tersebut berjalan dengan baik.



3. Jalankan program, dan kemudian klik beberapa kali tombol Increment sehingga angka yang ditampilkan berubah.
- a. Apakah program berjalan dengan baik serta menampilkan hal yang sama persis seperti yang dilakukan pada tahap nomor 4 sebelumnya?
Iya
- b. Di bagian kode program manakah saat ini proses increment dilakukan?
Di lambda onClick pada pemanggilan CounterScreen di dalam onCreate().
- c. Di bagian kode program manakah saat ini state Composable Text dan Composable Button "Increment" dideklarasikan?
State number kini dideklarasikan di onCreate(), tepat sebelum CounterScreen dipanggil.
4. Diasumsikan terdapat perubahan kebutuhan fungsional aplikasi yang memaksa terjadinya perubahan logika berjalannya program pada antar muka pengguna. Ubah kode program setelah tahap nomor 7 dilakukan sedemikian rupa sehingga:
- a. Tuliskan bagian kode program yang mendeklarasikan state dan pemanggilan fungsi Composable CounterScreen di dalam fungsi onCreate() setelah langkah nomor 8(a), 8(b), dan 8(c) dilakukan.

Kotlin

```
var number by remember { mutableStateOf(1) }
```

```
CounterScreen(  
    modifier = Modifier.padding(32.dp),  
    number = number,  
    label = "Double",  
    onClick = { number *= 2 }  
)
```

5. Jalankan program,

- a. Perhatikan dengan mengambil satu screenshot sebelum dan sesudah nilai variabel number digandakan.



- b. Di bagian manakah dari kode program pada percobaan tahap nomor 7 dan tahap nomor 8 dilakukan?

Keduanya dilakukan di onCreate(), yaitu pada state number dan pada argumen yang diberikan saat memanggil CounterScreen.

- c. Apakah ada perubahan yang dilakukan dari tahap nomor 7 dan tahap nomor 8 yang dilakukan di dalam fungsi Composable CounterScreen itu sendiri? Jika ada sebutkan perubahan yang dilakukan.

Tidak ada. Signature dan isi CounterScreen tetap sama.

6. Apa arti dari jawaban pada tahap nomor 9 terkait dengan manfaat dari melakukan *state hoisting* dalam perubahan kebutuhan fungsional aplikasi yang terkait dengan antar muka pengguna?

Ketika kebutuhan fungsional berubah, cukup bagian pemanggil yang disesuaikan, sedangkan komponen UI-nya tidak perlu disentuh. Dengan begitu CounterScreen bisa dipakai ulang dengan perilaku yang berbeda-beda, risiko munculnya bug baru di komponen UI lebih kecil, dan logika terpusat di satu tempat sebagai single source of truth.

Tugas Praktik

Lembar Kerja Praktik

1.1.1 Perubahan Konfigurasi

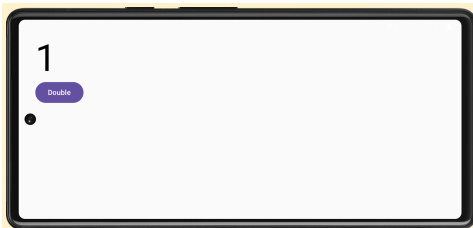
Pra-Percobaan

Aktivitas Percobaan

1. Jalankan program, dengan memastikan posisi perangkat pada posisi **portrait**, kemudian klik beberapa kali tombol "Double" sedemikian rupa sehingga angka yang ditampilkan tidak lagi angka 1 yang merupakan nilai awal yang diset pada variabel number saat program pertama kali dijalankan. Catat angka yang ditampilkan setelah tombol "Double" diklik beberapa kali, ambil screenshot tampilan antarmukanya.

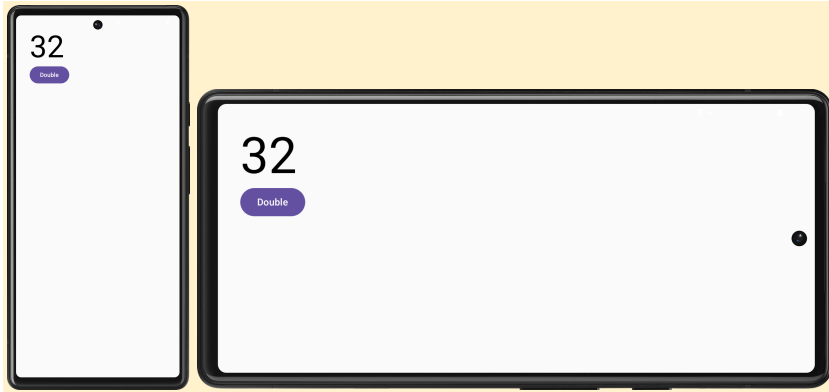


2. Lakukan perubahan konfigurasi dengan memutar posisi perangkat dari posisi **portrait** menjadi **landscape**, sehingga tampilan aplikasi akan disesuaikan oleh Android.
 - a. Catat angka yang ditampilkan kemudian ambil screenshot tampilan antarmuka aplikasi di posisi landscape.

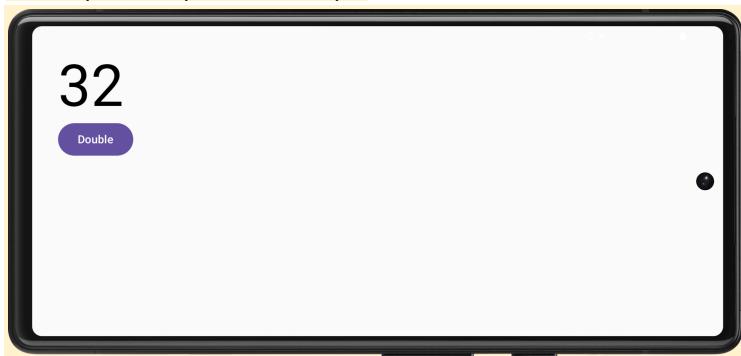


- b. Berapa angka yang ditampilkan? Mengapa terjadi demikian?
Angka kembali menjadi 1, karena rotasi layar termasuk perubahan konfigurasi, yang membuat Android menghancurkan lalu membuat ulang Activity. Komposisi ikut dibuat ulang dari awal. remember hanya menjaga nilai selama recomposition di dalam komposisi yang sama, sehingga nilainya hilang dan number diinisialisasi ulang menjadi 1.

Catat angka yang ditampilkan setelah tombol "Double" diklik beberapa kali, ambil screenshot tampilan antarmukanya.



3. Lakukan kembali perubahan konfigurasi dengan memutar posisi perangkat dari posisi *portrait* menjadi *landscape*, sehingga tampilan aplikasi akan disesuaikan oleh Android.
 - a. Catat angka yang ditampilkan kemudian ambil screenshot tampilan antar muka aplikasi di posisi *landscape*.



- b. Berapa angka yang ditampilkan? Mengapa terjadi demikian?
Angka tetap 32, karena `rememberSaveable` menyimpan nilai state ke dalam Bundle sebelum Activity dihancurkan. Setelah Activity dibuat ulang, nilai tersebut dipulihkan ke variabel `number`, sehingga angka tidak kembali ke nilai awal.
 - c. Apa fungsi dari konstruk `rememberSaveable` pada kode program Android yang membedakannya dari konstruk `remember`?
`remember` mempertahankan nilai hanya selama recomposition. `rememberSaveable` juga mempertahankannya ketika Activity dibuat ulang, misalnya karena rotasi layar, perubahan bahasa atau tema, dan ketika proses aplikasi dimatikan sistem untuk menghemat memori.

4. Beri contoh kasus nyata lain di mana penggunaan konstruk `rememberSaveable` ini tepat digunakan dalam aplikasi Android serta sebutkan manfaat yang diperoleh dari penerapannya!

Pemesanan tiket bioskop saat pengguna memilih kursi. Ketika pengguna sudah memilih beberapa kursi lalu memutar perangkat ke landscape, Activity dibuat ulang karena perubahan konfigurasi. Jika memakai `remember`, kursi yang dipilih hilang, tetapi dengan `rememberSaveable` pilihan tersebut tetap tersimpan dan tampil kembali di layar landscape. Manfaatnya, pengguna tidak perlu memilih ulang kursi sehingga proses pemesanan lebih nyaman.

Tugas Praktik

Lembar Kerja Praktik

1.1.2 LaunchedEffect

Pra-Percobaan 1

Aktivitas Percobaan

1. Jalankan program,
 - a. Pada saat pertama kali program dijalankan, tanpa mengklik *button* "Double" apa yang ditampilkan di dalam layar?

Number 1 is shown



- b. Mengapa pesan tersebut ditampilkan saat pertama kali aplikasi dijalankan tanpa ada interaksi apa pun dari pengguna?

LaunchedEffect dijalankan otomatis begitu composables pertama kali masuk ke komposisi, tanpa menunggu aksi pengguna

- c. Apa yang memicu pesan pada Snackbar tersebut ditampilkan? Pada bagian kode program yang mana?

Berjalan saat pertama kali masuk komposisi dan setiap kali nilai number berubah. Pemicunya pada bagian

`LaunchedEffect(number) { snackbarHostState.showSnackbar("Number $number is shown!") }` yang diletakkan di dalam Scaffold pada `onCreate()`.

2. Klik button "Double" satu kali,

a. Apa yang terjadi?

Angka berubah dari 1 menjadi 2 dan muncul Snackbar baru berisi "Number 2 is shown!"

b. Mengapa hal tersebut terjadi? Jelaskan!

Karena number dijadikan key pada LaunchedEffect. Ketika nilainya berubah, Compose membatalkan coroutine yang lama dan menjalankan coroutine baru dengan nilai terbaru, sehingga showSnackbar() dipanggil lagi.

Pra-Percobaan 2

Aktivitas Percobaan

1. Jalankan program,

a. Pada saat pertama kali program dijalankan, tanpa mengklik *button* "Double" apa yang ditampilkan di dalam layar?



c. Tunggu selama dua detik, kemudian periksa kembali apa yang ditampilkan pada layar! Apa yang tampil?

Angka berubah menjadi bilangan acak antara 100 dan 999, Snackbar menampilkan "Number 984 is shown!".

d. Apa yang memicu perubahan angka dan pesan yang ditampilkan pada Snackbar tersebut ditampilkan? Pada bagian kode program yang mana?

Angka berubah oleh `LaunchedEffect(Unit) { number = Repo.getData() }`. Kode ini memanggil `getData()`, yang menunggu 2 detik (`delay(2000)`) lalu mengembalikan angka acak, dan hasilnya disimpan ke `number`. Karena `number` adalah key pada `LaunchedEffect(number)`, perubahan tersebut memicu `showSnackbar()` berjalan lagi dan menampilkan pesan dengan angka baru.

3. Dari kedua percobaan terkait dengan side-effect dengan `LaunchedEffect`,
- a. Jelaskan apa perbedaan antara `LaunchedEffect(state)` dengan `LaunchedEffect(Unit)`?

Pada `LaunchedEffect(state)`, key-nya adalah state yang nilainya bisa berubah, sehingga blok kode dijalankan ulang setiap key berubah. Pada `LaunchedEffect(Unit)`, key-nya adalah konstanta yang tidak pernah berubah, sehingga blok kode tidak dijalankan ulang selama composible tetap berada dalam komposisi.

- b. Kapan operasi di dalam `LaunchedEffect(state)` dengan `LaunchedEffect(Unit)` dijalankan?

`LaunchedEffect(state)` dijalankan saat composible pertama kali masuk komposisi dan setiap kali nilai state berubah. Coroutine sebelumnya dibatalkan terlebih dahulu. `LaunchedEffect(Unit)` hanya dijalankan satu kali, yaitu saat composible pertama kali masuk komposisi, sehingga cocok untuk pekerjaan awal seperti mengambil data ketika layar dibuka.